

변경분 — signus 필사용 코드북 바뀐 파일은 통째로 실었습니다 · 초록 줄만 달라진 곳

기준점 2026-07-27 (c6a9bc6+수정중) → 2026-07-29 5aaedf+수정중 | 3개 파일 · +67 / -3 줄 · 총 18쪽 | 새 코드북 78쪽

변경된 파일 (괄호=쪽) pyproject.toml(1) · signus/pipeline.py(1) · signus/cli.py(10)

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · 붉은 점선 = 그 자리에서 몇 줄 사라짐 · L = 윗줄에 붙는 이어짐

pyproject.toml +2 · 코드북 2쪽 · 새로 쓸 줄 14-15

```

1 1  [build-system]
2 2  requires = ["setuptools>=68"]
3 3  build-backend = "setuptools.build_meta"
4 4
5 5  [project]
6 6  name = "signus"
7 7  version = "2.0.0"
8 8  description = "Blind PSK/QAM demodulator: auto parameter detection + constellation UI"
9 9  requires-python = ">=3.11"
10 10 dependencies = ["numpy>=1.26", "scipy>=1.11"]
11 11
12 12 [project.optional-dependencies]
13 13 dev = ["pytest>=8.0", "ruff>=0.4"]
14 + # 필사 인쇄물 발급기(tools/codebook.py) 전용. 분석기 본체는 여전히 numpy+scipy 둘뿐이다.
15 + tools = ["pygments>=2.17"]
16 # Optional JIT for the sequential equalizer/carrier loops (16-34x on those loops).
17 # Falls back to identical pure-numpy when absent. NOTE: numba pins numpy<2.5.
18 fast = ["numba>=0.60"]
19
20 [project.scripts]
21 signus = "signus.cli:main"
22
23 [tool.setuptools.packages.find]
24 include = ["signus*"]
25
26 [tool.setuptools.package-data]
27 signus = ["web/*"] # the served frontend: without this a pip install 404s the whole UI
28
29 [tool.pytest.ini_options]
30 pythonpath = ["."]
31 testpaths = ["tests"]
32 addopts = "-q"
33 markers = ["stretch: slow/marginal cases, opt-in"]
34
35 [tool.ruff]
36 target-version = "py311"
37 line-length = 100
38
39 [tool.ruff.lint]
40 select = ["E", "F", "I", "UP", "B", "SIM", "NPY"]

```

signus/pipeline.py +4 -2 · 코드북 39쪽 · 새로 쓸 줄 104-106, 433

```

1 1  """End-to-end blind demodulation. Two families:
2 2      linear (PSK/QAM): front-end -> carrier -> baud -> matched filter -> timing ->
3 3                          classify -> fine sync -> [equalizer rescue] -> quality
4 4      fsk (CPFSK/MSK): frequency discriminator (gated by constant envelope)
5 5  Classification runs BEFORE fine sync so the decision-directed loop knows the
6 6  constellation. Differential demapping is an option, not a detection: dqpsk and
7 7  qpsk share a constellation."""
8 8
9 9  import json
10 10 from dataclasses import dataclass, field
11 11
12 12 import numpy as np

```

```

13 13
14 14 from . import classify as cl
15 15 from . import dsp, triage
16 16 from .channelize import extract
17 17 from .chirp import analyze_chirp, is_chirp, sweeps_band
18 18 from .constellations import demap_bits, demap_diff_bits, mod_order
19 19 from .detect import Detection, detect
20 20 from .eq import equalize, equalize_fse
21 21 from .fsk import analyze_fsk, fsk_gate
22 22 from .sigio import Meta, parse_name, read
23 23 from .spectrum import spectrum, waterfall
24 24 from .sync import find_preamble
25 25
26 26 _BITS_CAP = 65536
27 27 _EQ_LOCK = 60.0 # below this a linear signal is worth an equalizer attempt
28 28 _EQ_GAIN = 3.0 # ...keep it only if lock improves by this much
29 29 _EQ_FLOOR = 50.0 # ...and clears this floor, so a wrong-mod fit is never locked in
30 30 _EQ_QAM_FLOOR = 60.0 # ...but a DENSE-QAM (32/64) eq fit must clear a higher bar: the DD loop c
    l, l
    an
31 31 #
    l, l
    , a
32 32 #
    l, l
    's
33 33 #
    l, l
    al.
34 34 _BAUD_GAIN = 5.0 # an in-band baud hypothesis must beat the global one by this lock
35 35 _SHORT_LOCK = 60.0 # below this, retry the baud line with fewer blocks (short-burst rescue)
36 36 _SYNC_LOCK = 60.0 # below this, try a repeated-preamble carrier/timing sync (packetized bursts
    l, l
    )
37 37 _ALIAS_MARGIN = 10.0 # a preamble carrier alias must beat the next by this lock, else ambiguous
    l, l
    .
38 38 #
    l, l
    lf
39 39 #
    l, l
    n
40 40 #
    l, l
    s it,
41 41 #
    l, l
    s.
42 42 _SYNC_ACCEPT = 78.0 # ...AND clear this absolute lock: below it correct/rotated aliases overlap
    l, l
    . A
43 43 #
44 44 #
    l, l
    8 band
45 45 #
    l, l
    s a
46 46 #
    l, l
    t 78.
47 47 _SYNC_ADIST = 1.5 # ...AND sit within this many alias steps of the coarse est_carrier fc -- a
48 48 #
    l, l
    s,
49 49 #
50 50 #
51 51 #
52 52 _DIFF_OF = {"bpsk": "dbpsk", "qpsk": "dqpsk"} # pi4dqpsk arrives as qpsk -> dqpsk
53 53
54 54
55 55 @dataclass
56 56 class Result:
57 57     meta: Meta

```

```

58 58     family: str                                # 'linear' | 'fsk'
59 59     burst: tuple[int, int]
60 60     fc: float
61 61     baud: float
62 62     mod: str
63 63     lock: float
64 64     symbols: np.ndarray                        # aligned symbols (linear) / normalized levels (fsk)
65 65     bits: np.ndarray
66 66     iq_corr: np.ndarray                        # exportable corrected baseband
67 67     burst_x: np.ndarray = field(repr=False, default=None) # for spectrum views
68 68     symmetry: int = 0
69 69     carrier_ambiguous: bool = False
70 70     baud_conf: float = 0.0
71 71     rolloff: float | None = None
72 72     h: float | None = None                    # FSK modulation index
73 73     evm: float = float("nan")
74 74     mer_db: float = float("nan")
75 75     occupied: int = 0
76 76     snr_est: float = float("nan")
77 77     eq_applied: bool = False
78 78     eq_mode: str | None = None                # 'sym' | 'fse'
79 79     alias_resolved: bool = False
80 80     baud_fallback: bool = False
81 81     bursts: list = field(default_factory=list)
82 82     burst_idx: int = 0
83 83     preamble: object = None                  # sync.Preamble when a repeated preamble drove the decode
84 84
85 85     def to_json(self, max_points: int = 6000, views: bool = True) -> dict:
86 86         z = np.nan_to_num(self.symbols)      # a dead/DC capture -> NaN symbols -> invalid JSON
87 87         if z.size > max_points: # keep TIME ORDER: the UI animates this sequence
88 88             z = z[np.linspace(0, z.size - 1, max_points).astype(int)]
89 89         det = {"fc": round(self.fc, 3), "baud": round(self.baud, 3), "mod": self.mod,
90 90             "symmetry": self.symmetry, "carrier_ambiguous": self.carrier_ambiguous,
91 91             "baud_conf": round(self.baud_conf, 1),
92 92             "alias_resolved": self.alias_resolved,
93 93             "baud_fallback": self.baud_fallback}
94 94         det["rolloff"] = None if self.rolloff is None else round(self.rolloff, 3)
95 95         rf = self.meta.rf_center              # real RF = capture centre + baseband fc
96 96         det["rf_hz"] = None if rf is None else round(rf + self.fc, 3)
97 97         if self.h is not None:
98 98             det["h"] = round(self.h, 3)
99 99         if self.preamble is not None:         # a repeated preamble drove the sync
100 100             p = self.preamble
101 101             det["preamble"] = {"period": p.period, "cfo_hz": round(p.cfo_hz, 1),
102 102                 "conf": round(p.conf, 2), "start": p.start, "end": p.end}
103 103         doc = {
104 +         "fs": self.meta.fs, "fmt": self.meta.fmt, "dtype": self.meta.dtype, # dtyp
105 +         "e 은 한 줄",
106 +         "rf_center": self.meta.rf_center,      # 보고에 필수: lock 0 의 1순위 용의자다
107 107         "family": self.family, "n_samples": int(self.iq_corr.size),
108 108         "burst": {"start": self.burst[0], "end": self.burst[1]},
109 109         "bursts": [{"start": s, "end": e} for s, e in self.bursts],
110 110         "burst_idx": self.burst_idx,
111 111         "detected": det,
112 112         "quality": {"lock": _r(self.lock, 1) or 0.0, "mer_db": _r(self.mer_db),
113 113             "evm": _r(self.evm, 4)},
114 114         "snr_est_db": _r(self.snr_est),
115 115         "eq": {"applied": self.eq_applied, "mode": self.eq_mode},
116 116         "constellation": {"i": np.round(z.real, 4).tolist(),

```

```

115 117         "q": np.round(z.imag, 4).tolist()},
116 118         "bits": "".join(map(str, self.bits[:_BITS_CAP])),
117 119     }
118 120     if views and self.burst_x is not None:
119 121         doc["spectrum"] = spectrum(self.burst_x, self.meta.fs)
120 122         doc["waterfall"] = waterfall(self.burst_x, self.meta.fs)
121 123     return doc
122 124
123 125     def save_iq(self, path: str) -> None:
124 126         np.column_stack([self.iq_corr.real, self.iq_corr.imag]).astype("<f4").tofile(path)
125 127
126 128     def save_symbols(self, path: str) -> None:
127 129         np.save(path, self.symbols.astype(np.complex64))
128 130
129 131     def save_bits(self, path: str, packed: bool = False) -> None:
130 132         if packed:
131 133             np.packbits(self.bits).tofile(path)
132 134         else:
133 135             with open(path, "w") as fh:
134 136                 fh.write("".join(map(str, self.bits)))
135 137
136 138     def save_report(self, path: str) -> None:
137 139         with open(path, "w") as fh:
138 140             json.dump(self.to_json(), fh, indent=1)
139 141
140 142
141 143     def _r(v: float, nd: int = 2) -> float | None:
142 144         return None if v is None or not np.isfinite(v) else round(float(v), nd)
143 145
144 146
145 147     def _fine(z: np.ndarray, mod: str) -> np.ndarray:
146 148         """Bulk-phase removal, decision-directed carrier loop, final rotation lock."""
147 149         return cl.align(dsp.ddsync(cl.align(z, mod), mod), mod)
148 150
149 151
150 152     def _blocks(n: int) -> int:
151 153         """M-th-power spectra are averaged INCOHERENTLY, so a weak tone (high symmetry,
152 154         low SNR) wants fewer, longer segments; only long records need more for
153 155         stationarity. Worth ~2 dB at the 8PSK edge over a fixed blocks=4."""
154 156         return int(np.clip(n // 30000, 1, 8))
155 157
156 158
157 159     def _demod(xd: np.ndarray, fs: float, baud: float, symmetry: int) -> tuple:
158 160         """Matched filter + timing + classify + fine sync at one baud hypothesis."""
159 161         alpha = dsp.est_rolloff(xd, fs, baud)
160 162         ym = dsp.matched(dsp.to_sps(xd, fs, baud), 4, alpha)
161 163         raw = dsp.timing(ym, 4)
162 164         mod = cl.classify(raw, symmetry)
163 165         # align BEFORE ddsync: the coarse carrier leaves only micro-Hz residual, so removing
164 166         # the bulk phase first kills the loop transient that would corrupt the first symbols
165 167         syms = _fine(raw, mod)
166 168         return alpha, ym, raw, mod, syms, cl.quality(syms, mod)
167 169
168 170
169 171     def _rescue(eq_fn, z: np.ndarray, seed_mod: str, symmetry: int) -> tuple:
170 172         """Equalize with a seed constellation, then let the OPENED eye pick the mod:
171 173         heavy ISI corrupts the ring test and even the 2/8 symmetry vote, so re-classify
172 174         with the estimated symmetry AND the neutral order-4 gate, keep the best lock."""
173 175         z1 = eq_fn(z, seed_mod)
174 176         cand = []

```

```

175 177 for m in {cl.classify(z1, symmetry), cl.classify(z1, 4)}: # symmetry too, per the docstrin
176 178     g
177 179     s = _fine(z1 if m == seed_mod else eq_fn(z, m), m)
178 180     cand.append((cl.quality(s, m), s, m))
179 181 best = max(cands, key=lambda c: c[0].lock)
180 182 # Occam de-fold for the PSK point-doubling: CMA is modulus-only, so a bpsk signal under a
181 183 # multipath echo ALSO fits a qpsk ring at a marginally higher lock (phantom 2->4 points).
182 184 # A bpsk candidate that clears the accept floor cannot be an over-fit of qpsk, so prefer it.
183 185 # Only fires when symmetry==2 put bpsk in the set; a genuine qpsk reads symmetry 4 -> the
184 186 # candidate set is the qpsk singleton -> this branch is unreachable and it stays untouched.
185 187 lo = [c for c in cands if c[2] == "bpsk" and c[0].lock >= _EQ_FLOOR]
186 188 if lo and best[2] == "qpsk":
187 189     return lo[0]
188 190 return best
189 191
190 192 def analyze(x: np.ndarray, meta: Meta, diff: bool = False,
191 193     burst: int | None = None) -> Result:
192 194     """Run the blind chain. `diff` demaps phase transitions (D-BPSK/D-QPSK);
193 195     `burst` selects one detected burst (default: the most energetic)."""
194 196     if x.size:
195 197         # pathological UNIFORM scale must be fixed BEFORE analytic(): its magnitude sanitizer
196 198         # zeroes samples above 1e150 sample-WISE, so a whole capture near/over that ceiling was
197 199         # wiped wholesale (garbage mods, lock=nan at 1e154) before the rms-normalize below could
198 200         # ever help. The 99th percentile is robust to spike outliers (a single corrupt 1e300
199 201         # sample leaves it at signal scale -> untouched here, still zeroed sample-wise later);
200 202         # normal captures sit far inside (1e-100, 1e100) -> untouched -> sweep byte-identical.
201 203         scale = float(np.percentile(np.abs(x[:16]), 99)) # strided: scale detection needs the
202 204         # BULK magnitude, not every sample -- 16x cheaper on a large direct capture
203 205         if np.isfinite(scale) and scale > 0 and not (1e-100 < scale < 1e100):
204 206             x = x / scale
205 207     x = dsp.analytic(x)
206 208     x = x - x.mean() # DC block: LO leakage otherwise corrupts every estimator
207 209     if x.size < 64: # empty / trivially short: nothing to estimate, and it crashes downstream
208 210         raise ValueError(f"신호가 너무 짧습니다 ({x.size} 샘플)")
209 211     rms = float(np.sqrt(np.mean(np.abs(x) ** 2)))
210 212     if rms and (rms < 1e-6 or rms > 1e6): # only PATHOLOGICAL scale: normalize so the scale-var
211 213         x = x / rms # spots (fsk_gate's CV epsilon, est_carrier's x**p ov
212 214         # er/
213 215         # underflow) stay invariant. Normal captures rms~0(1
214 216         # untouched -> the acceptance sweep is byte-identical
215 217         .
216 218     bursts = dsp.find_bursts(x, meta.fs)
217 219     if burst is None or not 0 <= burst < len(bursts):
218 220         burst = int(np.argmax([np.sum(np.abs(x[s:e]) ** 2) for s, e in bursts]))
219 221     s, e = bursts[burst]
220 222     xb = x[s:e] if e - s >= 64 else x
221 223
222 224     # chirp gate -- same conjunctive tie-break as triage.family, so direct analyze / survey_web
223 225     # single mode agree with survey's label: an FMCW/CSS sweep trips fsk_gate (bimodal IF) and
224 226     # would force-demodulate into confident garbage FSK. sweeps_band keeps real FSK (0/222 pass)
225 227     # out of this branch, so only a genuine band-sweep is refused -- never demodulated.
226 228     if is_chirp(xb, meta.fs) and sweeps_band(xb, meta.fs):
227 229         info = analyze_chirp(xb, meta.fs)
228 230         kind = f"LoRa 추정 SF{info['sf']}" if info["sf"] else f"μ={info['mu']:.3g} Hz/s"
229 231         raise ValueError(f"처프(FMCW/CSS) 신호입니다 ({kind}, bw≈{info['bw']:.3g} Hz) - "

```

```

230 232 if fsk_gate(xb, meta.fs):
231 233     r = analyze_fsk(xb, meta.fs)
232 234     sym = np.asarray(r["symbols"], dtype=np.complex128)
233 235     return Result(meta, "fsk", (s, e), r["fc"], r["baud"], r["mod"], r["lock"],
234 236                     sym, r["bits"], xb, burst_x=xb, h=r["h"],
235 237                     bursts=bursts, burst_idx=burst)
236 238
237 239 fc0, symmetry, _ = dsp.est_carrier(xb, meta.fs, blocks=_blocks(xb.size))
238 240 fc = dsp.resolve_alias(xb, meta.fs, fc0, symmetry)
239 241 amb = abs(symmetry * fc) > 0.4 * meta.fs
240 242 xd = dsp.mix(xb, meta.fs, fc)
241 243
242 244 baud, conf = dsp.est_baud(xd, meta.fs)
243 245 fell = False
244 246 alpha, ym, raw, mod, syms, q = _demod(xd, meta.fs, baud, symmetry)
245 247 bw = dsp.occupied_bw(xd, meta.fs)
246 248 if bw and not (0.72 * bw <= baud <= 1.05 * bw):
247 249     # the global |x|^2 peak disagrees with the occupied band: below alpha~0.1
248 250     # (and under harsh channels) data junk outruns the true line. Demod the
249 251     # in-band hypotheses too and let lock decide, with a clear margin.
250 252     for lo in (0.80, 0.67):
251 253         b2, c2 = dsp.est_baud(xd, meta.fs, lo=lo * bw, hi=1.02 * bw)
252 254         if abs(b2 - baud) <= 0.01 * b2:
253 255             continue
254 256         t = _demod(xd, meta.fs, b2, symmetry)
255 257         if t[5].lock > q.lock + _BAUD_GAIN:
256 258             alpha, ym, raw, mod, syms, q = t
257 259             baud, conf, fell = b2, c2, True
258 260
259 261 if q.lock < _SHORT_LOCK:
260 262     # short / low-SNR burst rescue: the default 4-block |x|^2 line splits the record
261 263     # too finely and locks a junk peak. Fewer, longer blocks give a stronger line (the
262 264     # carrier _blocks logic, applied to baud). Run both and keep only a clearly-better
263 265     # lock -- so a long, already-locked signal (CORE) never enters here and is untouched.
264 266     for nb in (2, 1):
265 267         b2, c2 = dsp.est_baud(xd, meta.fs, blocks=nb)
266 268         if abs(b2 - baud) <= 0.01 * b2:
267 269             continue
268 270         t = _demod(xd, meta.fs, b2, symmetry)
269 271         if t[5].lock > q.lock + _BAUD_GAIN:
270 272             alpha, ym, raw, mod, syms, q = t
271 273             baud, conf, fell = b2, c2, True
272 274
273 275 eq_applied, eq_mode, pre = False, None, None
274 276 if q.lock < _EQ_LOCK: # ISI (multipath) is what a phase-only loop cannot fix
275 277     # CMA is modulus-only, so it opens the eye without knowing the constellation.
276 278     qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
277 279     mode = "sym"
278 280     if qe.lock < max(_EQ_FLOOR, q.lock + _EQ_GAIN):
279 281         # symbol-spaced FIR could not invert the channel; retry T/2-spaced on
280 282         # the clock-tracked 2 sps stream (unaliaised spectrum, longer inverse)
281 283         q2, s2, m2 = _rescue(lambda z, m: equalize_fse(z, m),
282 284                             dsp.timing(ym, 4, out=2), mod, symmetry)
283 285         if q2.lock > qe.lock:
284 286             qe, se, me, mode = q2, s2, m2, "fse"
285 287     floor = _EQ_QAM_FLOOR if me in ("32qam", "64qam") else _EQ_FLOOR
286 288     if qe.lock > q.lock + _EQ_GAIN and qe.lock >= floor:
287 289         syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
288 290 elif mod in ("16qam", "32qam", "64qam"):
289 291     # confident square-QAM at high lock: a benign post-echo can fold a LOWER-order signal

```

```

290 292 # onto a QAM lattice with a clean-looking eye, so the low-lock rescue above never fires.
291 293 # Equalize once and accept ONLY if a strictly lower order emerges -- verified never to
292 294 # demote a genuine QAM (0/36 across 16/32/64qam x snr x seeds), so real QAM is untouched
    ,
293 295 qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
294 296 mode = "sym"
295 297 if mod_order(me) < mod_order(mod) and qe.lock < _EQ_FLOOR:
296 298     # a lower order emerged but the symbol-spaced eye stayed shut (echo > ~2 symbols);
297 299     # retry T/2 fractionally-spaced. Only runs once a de-fold is INDICATED, so a genuine
298 300     # QAM (no lower order) never pays for the FSE.
299 301     qe, se, me = _rescue(lambda z, m: equalize_fse(z, m),
300 302         dsp.timing(ym, 4, out=2), mod, symmetry)
301 303     mode = "fse"
302 304 if mod_order(me) < mod_order(mod) and qe.lock >= _EQ_FLOOR:
303 305     syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
304 306
305 307 if q.lock < _SYNC_LOCK:
306 308     # packetized-burst rescue -- LAST, after the equalizer: a repeated preamble pins the
307 309     # carrier/baud/timing from only a few symbols, where the blind M-th-power estimator (nee
    ,
308 310     # many symbols to average) fails. Runs only if the eq rescue above did NOT lift the lock
    ,
309 311     # a multipath signal (which the eq handles, and whose ISI can fake a short period) never
310 312     # reaches here. Detect the preamble, mix by its CFO, demod the DATA past it, keep-best -
    ,
311 313     # random / no-preamble signal yields no plateau or a CFO that does not improve lock an
    ,
312 314     # dropped (the sweep stays byte-identical). The preamble period P = L*sps supplies the b
    ,
313 315     # (fs*L/P per block length L), the phase slope gives the carrier (+-fs/P resolves the L-
    ,
314 316     # alias); every PSK/QAM symmetry is tried -- the right (carrier, baud, sym) locks clean.
315 317     ps = find_preamble(xb, meta.fs, baud_hint=baud)
316 318     if ps is not None:
317 319         # The Schmidl-Cox CFO is ambiguous modulo fs/P = baud/L: an M-PSK carrier off by bau
    ,
318 320         # rotates a whole constellation position per symbol, so the eye still looks locked b
    ,
319 321         # the bits shift -> a confident WRONG decode if the wrong alias is trusted. The coar
    ,
320 322         # M-th-power fc lands within ~1 alias step of the truth even on these short bursts
    ,
321 323         # CENTRE the alias scan on it (k0) and sweep +-3 steps -- this reaches the correct a
    ,
322 324         # for any offset (not just |fc|<baud/2); keep-best over them picks the cleanest eye.
323 325         step = meta.fs / ps.period
324 326         k0 = round((fc - ps.cfo_hz) / step)
325 327         cands = []
326 328         for b2 in sorted({round(meta.fs * L / ps.period) for L in (3, 4, 6, 8)}):
327 329             for k in range(k0 - 3, k0 + 4):
328 330                 cfo = ps.cfo_hz + k * step
329 331                 data = dsp.mix(xb, meta.fs, cfo)[ps.end:]
330 332                 if data.size < 256:
331 333                     continue
332 334                 for sm in (2, 4, 8):
333 335                     t = _demod(data, meta.fs, float(b2), sm)
334 336                     cands.append((t[5].lock, cfo, float(b2), sm, t))
335 337                 cands.sort(key=lambda c: -c[0])
336 338         # Adjacent aliases both look like clean M-PSK, so lock alone can pick the WRONG on
    ,
    ,
    e by a

```

```

337 339 # hair. Require the winner to beat the best DIFFERENT-carrier candidate by a clear g
    L, L ap;
338 340 # otherwise the alias is genuinely ambiguous -> leave the honest low-lock result rat
    L, L her
339 341 # than emit a confident wrong decode. (Same-carrier baud/sym variants do not compete
    L, L .)
340 342 # PRECISION GATE: the alias ambiguity (carrier off by baud/L) is blind-ambiguous -
    L, L - a
341 343 # rotated alias still locks AS HIGH AS the truth (both are clean M-PSK), so lock alo
    L, L ne
342 344 # cannot separate them (a rotated 8psk alias was measured locking 78 with ber 0.38)
    L, L . Two
343 345 # conjunctive conditions favour precision over recall: (1) a strong absolute lock, a
    L, L nd
344 346 # (2) the winner sits within _SYNC_ADIST alias steps of the coarse est_carrier fc -
    L, L - a
345 347 # baud/L rotation is >=1 step off the true carrier, so a far-from-anchor winner is a
346 348 # rotation. Genuine recoveries have a near-anchor carrier (adist<=0.5, lock 88+); o
    L, L n the
347 349 # hardest bursts the anchor itself is unreliable -> both conditions fail -> refuse.
348 350 if cands and cands[0][0] >= _SYNC_ACCEPT and cands[0][0] > q.lock + _EQ_GAIN:
349 351     top = cands[0]
350 352     other = next((c for c in cands if abs(c[1] - top[1]) > step / 2), None)
351 353     anchored = abs(top[1] - fc) <= _SYNC_ADIST * step
352 354     if anchored and (other is None or top[0] - other[0] >= _ALIAS_MARGIN):
353 355         alpha, ym, raw, mod, syms, q = top[4]
354 356         fc, baud, symmetry = top[1], top[2], top[3]
355 357         eq_applied, eq_mode, pre = False, None, ps
356 358         # diagnostics must describe the ACCEPTED decode, not the abandoned blind
357 359         # attempt: the carrier came from the preamble (not resolve_alias -> fc0=fc
358 360         # keeps alias_resolved False), the baud from the preamble period (no
359 361         # spectral line -> conf 0, no fallback), and ambiguity follows the new fc.
360 362         fc0, conf, fell = fc, 0.0, False
361 363         amb = abs(symmetry * fc) > 0.4 * meta.fs
362 364
363 365 bits = demap_diff_bits(syms, _DIFF_OF[mod]) if diff and mod in _DIFF_OF \
364 366     else demap_bits(syms, mod)
365 367 return Result(meta, "linear", (s, e), fc, baud, mod, q.lock, syms, bits, ym,
366 368     burst_x=xb, symmetry=symmetry, carrier_ambiguous=amb, baud_conf=conf,
367 369     rolloff=alpha, evm=q.evm, mer_db=q.mer_db, occupied=q.occupied,
368 370     snr_est=cl.snr_m2m4(syms, mod), eq_applied=eq_applied, eq_mode=eq_mode,
369 371     alias_resolved=abs(fc - fc0) > 1.0, baud_fallback=fell,
370 372     bursts=bursts, burst_idx=burst, preamble=pre)
371 373
372 374
373 375 def analyze_file(path: str, fs: float | None = None, fmt: str | None = None,
374 376     dtype: str | None = None, endian: str | None = None,
375 377     bitrev: bool | None = None, diff: bool = False,
376 378     burst: int | None = None, rf: float | None = None) -> Result:
377 379     """Analyze a file; explicit args override SigMF/filename tokens."""
378 380     from .sigio import parse_sigmf
379 381     m = parse_sigmf(path) or parse_name(path)
380 382     meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
381 383         endian or m.endian, m.bitrev if bitrev is None else bitrev,
382 384         rf_center=rf if rf is not None else m.rf_center)
383 385     x, meta = read(path, meta)
384 386     return analyze(x, meta, diff, burst)
385 387
386 388
387 389 # --- wideband survey: many emitters in one capture -----

```

```

388 390
389 391 @dataclass
390 392 class Emitter:
391 393     detection: Detection
392 394     kind: str                # linear|fsk|chirp|analog|tone|tooshort|error
393 395     abs_fc: float           # emitter carrier vs capture centre (Hz)
394 396     result: Result | None = None # demod result for digital kinds
395 397     info: dict | None = None    # characterization for non-digital kinds (e.g. chirp params)
396 398
397 399     def to_json(self) -> dict:
398 400         d = self.detection
399 401         doc = {"kind": self.kind, "abs_fc": round(self.abs_fc, 3),
400 402             "det": {"fc": round(d.fc, 3), "bw": round(d.bw, 3),
401 403                 "t0": d.t0, "t1": d.t1, "snr_db": _r(d.snr_db),
402 404                 "baud_hint": round(d.baud_hint, 1)}}
403 405         if self.info is not None:
404 406             doc["info"] = self.info
405 407         if self.result is not None:
406 408             r = self.result
407 409             doc.update(mod=r.mod, baud=round(r.baud, 3), lock=round(r.lock, 1),
408 410                 mer_db=_r(r.mer_db), evm=_r(r.evm, 4), family=r.family)
409 411         return doc
410 412
411 413     def to_detail(self, rf_center: float | None = None) -> dict:
412 414         """Web drill-down payload: the box summary plus, for a digital emitter, the
413 415         full analyze result so the UI reuses its single-signal detail view verbatim.
414 416         The channel is demodulated at baseband (no rf_center), and r.fc is only the
415 417         residual within-channel offset -- so the emitter's absolute RF is the capture
416 418         centre plus abs_fc (the full offset from capture centre), injected here."""
417 419         doc = self.to_json()
418 420         if self.result is not None:
419 421             doc["result"] = self.result.to_json()
420 422             if rf_center is not None:
421 423                 doc["result"]["detected"]["rf_hz"] = round(rf_center + self.abs_fc, 3)
422 424         return doc
423 425
424 426
425 427 @dataclass
426 428 class Survey:
427 429     meta: Meta
428 430     emitters: list[Emitter] = field(default_factory=list)
429 431
430 432     def to_json(self) -> dict:
431 433     433 +     return {"fs": self.meta.fs, "fmt": self.meta.fmt, "dtype": self.meta.dtype,
432 434         "n_emitters": len(self.emitters),
433 435         "emitters": [e.to_json() for e in self.emitters]}
434 436
435 437
436 438 def survey(x: np.ndarray, meta: Meta, *, diff: bool = False, **detect_kw) -> Survey:
437 439     """Detect every emitter in a wideband capture, extract each to its own baseband
438 440     channel, and demodulate the digital ones with the unchanged single-signal chain.
439 441     A capture holding one signal that fills the band collapses to a single emitter."""
440 442     xa = dsp.analytic(x)
441 443     xa = xa - xa.mean()
442 444     emitters = []
443 445     for d in detect(xa, meta.fs, **detect_kw):
444 446         ch, fs_ch = extract(xa, meta.fs, d)
445 447         if ch.size < 256:
446 448             emitters.append(Emitter(d, "tooshort", d.fc))
447 449         continue

```

```

448 450 kind = triage.family(ch, fs_ch)
449 451 if kind in ("linear", "fsk"):
450 452     # triage only gates digital vs not; analyze's own family is authoritative.
451 453     # Isolate each channel: a degenerate one must not abort the whole survey.
452 454     try:
453 455         r = analyze(ch, Meta(fs_ch, "iq", "f32", "le", False), diff=diff)
454 456         emitters.append(Emitter(d, r.family, d.fc + r.fc, r))
455 457     except Exception:
456 458         emitters.append(Emitter(d, "error", d.fc))
457 459     else:
458 460         info = analyze_chirp(ch, fs_ch) if kind == "chirp" else None
459 461         emitters.append(Emitter(d, kind, d.fc, info=info))
460 462 return Survey(meta, emitters)
461 463
462 464
463 465 def survey_web(x: np.ndarray, meta: Meta, *, diff: bool = False) -> dict:
464 466     """Web survey payload for /api/survey. When detect sees <=1 emitter the capture
465 467     is effectively one signal -> 'single' mode returns the UNCHANGED direct analyze()
466 468     result (correct fc, matches /api/analyze). Otherwise 'survey' mode adds a whole-
467 469     capture overview waterfall and every emitter's drill-down detail. Additive: does
468 470     not alter survey()/Survey/Emitter, so CLI + existing tests are unaffected."""
469 471     xa = dsp.analytic(x)
470 472     xa = xa - xa.mean()
471 473     if len(detect(xa, meta.fs)) <= 1:
472 474         r = analyze(x, meta, diff=diff)
473 475         out = {"mode": "single", "result": r.to_json()}
474 476         if len(r.bursts) > 1: # multi-burst signal -> whole-record map for burst selection
475 477             out["overview"] = {"n": int(xa.size), "fs": meta.fs,
476 478                                "waterfall": waterfall(xa, meta.fs)}
477 479         return out
478 480     sv = survey(x, meta, diff=diff)
479 481     return {"mode": "survey", "fs": meta.fs, "fmt": meta.fmt, "rf_center": meta.rf_center,
480 482            "overview": {"fs": meta.fs, "n": int(xa.size), "spectrum": spectrum(xa, meta.fs),
481 483                        "waterfall": waterfall(xa, meta.fs)},
482 484            "emitters": [e.to_detail(meta.rf_center) for e in sv.emitters]}
483 485
484 486
485 487 def survey_file(path: str, fs: float | None = None, fmt: str | None = None,
486 488                 dtype: str | None = None, endian: str | None = None,
487 489                 bitrev: bool | None = None, diff: bool = False,
488 490                 rf: float | None = None) -> Survey:
489 491     """Survey a capture file; explicit args override SigMF/filename tokens."""
490 492     from .sigio import parse_sigmf
491 493     m = parse_sigmf(path) or parse_name(path)
492 494     meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
493 495                endian or m.endian, m.bitrev if bitrev is None else bitrev,
494 496                rf_center=rf if rf is not None else m.rf_center)
495 497     x, meta = read(path, meta)
496 498     return survey(x, meta, diff=diff)

```

signus/cli.py +61 -1 · 코드북 48쪽 · 새로 쓸 줄 4, 6, 29-79, 259-260, 290-291, 327-328, 389-390

```

1 1 """CLI: analyze FILE / gen / sweep (acceptance harness) / serve."""
2 2
3 3 import argparse
4 + import re
4 5 import sys
6 + from zlib import crc32
5 7
6 8 import numpy as np
7 9

```

```

8 10 from .constellations import (
9 11     GEN_MODS,
10 12     MODS,
11 13     bit_labels,
12 14     demap_bits,
13 15     demap_diff_bits,
14 16     family,
15 17     ideal_points,
16 18     mod_order,
17 19     mod_symmetry,
18 20 )
19 21 from .gen import GenParams, generate, save
20 22 from .pipeline import Result, analyze, analyze_file, survey_file
21 23 from .sigio import Meta, sidecar_read
22 24
23 25 _DTYPE_CHOICES = ("i8", "u8", "i16", "u16", "f32", "f64")
24 26
25 27 _SNR = {"bpsk": 14, "qpsk": 14, "8psk": 18, "16qam": 22, "64qam": 28}
26 28 _SNR_LOW = {"bpsk": 6, "qpsk": 8, "8psk": 12, "16qam": 16, "64qam": 22}
29 +
30 +
31 + # --- 손으로 옮기는 한 줄 -----
32 + # 실신호 장비는 인터넷도 클립보드도 없다. 결과는 사람이 화면을 보고 받아쳐서 나온다. 그래서
33 + # 이 블록은 반드시 여기(필사 대상)에 있어야 한다 -- tools/ 에 두면 그 도구부터 필사해야 하는
34 + # 본말전도가 된다. 줄 끝의 검출 코드가 "옮기다 한 글자 틀림"을 잡는다.
35 +
36 + _ALPHA = "0123456789abcdefghijklmnopqrstuvwxyz" # 0/o, 1/l/i 처럼 화면에서 헷갈리는 글자를 뺀 32자
37 + _BRIEF_FLAGS = [("eq", lambda d: d["eq"]["applied"]),
38 +                 ("al", lambda d: d["detected"]["alias_resolved"]),
39 +                 ("fb", lambda d: d["detected"]["baud_fallback"]),
40 +                 ("amb", lambda d: d["detected"]["carrier_ambiguous"]),
41 +                 ("pre", lambda d: "preamble" in d["detected"])]
42 +
43 +
44 + def check_code(text: str) -> str:
45 +     """받아친 줄의 오타 검출 코드 (4글자 = 20비트). 공백은 '한 칸으로 줄이되 없애지는'
46 +     않는다 -- 전부 지우면 'fs1000000 16qam' 과 'fs10000001 6qam' (샘플레이트 10배!) 이 같은
47 +     코드가 되어, 값이 바뀌는 오타 32종이 조용히 통과했다. 대소문자와 줄 간격은 계속 무시한다."""
48 +     body = re.sub(r"\s+", " ", re.sub(r"#[0-9a-z]{4}\b", "", text.lower())).strip()
49 +     n = crc32(body.encode())
50 +     return "".join(_ALPHA[(n >> (5 * i)) & 31] for i in (3, 2, 1, 0))
51 +
52 +
53 + def brief(doc: dict, mode: str) -> str:
54 +     """손으로 옮기는 한 줄 + 검출 코드. Result.to_json() 디렉터리에서 만든다 -- 객체
55 +     속성을 직접 포맷하면 to_json 이 이미 한 반올림과 두 번 겹쳐 장비와 맥이 갈린다."""
56 +     head = f"sig2 {mode} fs{doc['fs']:.0f} {doc['fmt']}--{doc['dtype']}"
57 +     if mode == "sv":
58 +         lines = [f"{head} n{doc['n_emitters']}"]
59 +         for i, e in enumerate(doc["emitters"][:12]):
60 +             baud = f" bd{e['baud']:.0f}" if e.get("baud") else ""
61 +             lock = f" lk{e['lock']:.0f}" if e.get("lock") is not None else ""
62 +             lines.append(f"{i} fc{e['abs_fc']:.0f}{baud}{lock} {e.get('mod') or e['kind']}")
63 +         if len(doc["emitters"]) > 12:
64 +             lines.append(f"...{len(doc['emitters']) - 12}개 생략")
65 +     else:
66 +         d, q = doc["detected"], doc["quality"]
67 +         p = [head, d["mod"], f"fc{d['fc']:.0f}", f"bd{d['baud']:.0f}", f"lk{q['lock']:.0f}"]
68 +         if q["mer_db"] is not None:
69 +             p.append(f"mer{q['mer_db']:.1f}")

```

```

70 +         if d.get("h") is not None:                                # FSK 는 롤오프 대신 변조지수
71 +             p.append(f'h{d["h"]:.2f}')
72 +         elif d["rolloff"] is not None:
73 +             p.append(f'rl{d["rolloff"]:.2f}')
74 +         if len(doc["bursts"]) > 1:
75 +             p.append(f'b{doc["burst_idx"] + 1}/{len(doc["bursts"])}')
76 +         p += [f for f, get in _BRIEF_FLAGS if get(doc)]
77 +         lines = [" ".join(p)]
78 +         body = "\n".join(lines)
79 +         return f'{body} #{check_code(body)}'

27 80
28 81
29 82 # --- BER scoring (blind reception leaves rotation/conjugation/offset ambiguity) ---
30 83
31 84 def ber(symbols: np.ndarray, mod: str, tx_bits: np.ndarray) -> float:
32 85     """Min BER over conjugation, M-fold rotation, and symbol offset (found by
33 86     correlating against the reconstructed TX symbol stream)."""
34 87     k = mod_order(mod).bit_length() - 1
35 88     sym = mod_symmetry(mod)
36 89     inv = np.empty(mod_order(mod), dtype=int)
37 90     inv[bit_labels(mod)] = np.arange(mod_order(mod))
38 91     labels = (tx_bits.reshape(-1, k) << np.arange(k - 1, -1, -1)).sum(1)
39 92     tx = ideal_points(mod)[inv[labels]]
40 93
41 94     best = 1.0
42 95     for z in (symbols, np.conj(symbols)):
43 96         # |correlation| is rotation-invariant -> find the symbol offset first
44 97         offs = range(-32, 33)
45 98         c = [np.abs(np.vdot(*_overlap(z, tx, o))) for o in offs]
46 99         o = list(offs)[int(np.argmax(c))]
47 100        a, b = _overlap(z, tx, o)
48 101        if a.size < 100:
49 102            continue
50 103        tx_b = demap_bits(b, mod)
51 104        for r in range(sym):
52 105            rx_b = demap_bits(a * np.exp(-2j * np.pi * r / sym), mod)
53 106            best = min(best, float(np.mean(rx_b != tx_b)))
54 107        return best
55 108
56 109
57 110 def _overlap(rx: np.ndarray, tx: np.ndarray, off: int, crop: int = 20) -> tuple:
58 111     """Aligned overlapping slices of rx[i] vs tx[i+off], edges cropped."""
59 112     i0 = max(0, -off) + crop
60 113     i1 = min(rx.size, tx.size - off) - crop
61 114     if i1 <= i0:
62 115         return rx[:0], tx[:0]
63 116     return rx[i0:i1], tx[i0 + off:i1 + off]
64 117
65 118
66 119 def ber_bits(rx: np.ndarray, tx: np.ndarray, k: int, span: int = 8, crop: int = 20) -> float:
67 120     """BER of an absolute bit stream over whole-SYMBOL shifts. No rotation search:
68 121     differential transitions and FSK levels carry no phase ambiguity."""
69 122     a, b = rx[: rx.size // k * k].reshape(-1, k), tx[: tx.size // k * k].reshape(-1, k)
70 123     best = 1.0
71 124     for sh in range(-span, span + 1): # sh = symbol offset of rx within tx
72 125         i0, i1 = max(0, -sh) + crop, min(len(a), len(b) - sh) - crop
73 126         if i1 - i0 >= 400:
74 127             best = min(best, float(np.mean(a[i0:i1] != b[i0 + sh:i1 + sh])))
75 128     return best
76 129

```

```

77 130
78 131 # --- sweep: the acceptance grid -----
79 132
80 133 # constellation-identical to their parent (differential is a demap option, not a class)
81 134 _EXPECT = {"dbpsk": "bpsk", "dqpsk": "qpsk", "pi4dqpsk": "qpsk"}
82 135 _DIFF = {"dbpsk": "dbpsk", "dqpsk": "dqpsk", "pi4dqpsk": "dqpsk"}
83 136
84 137
85 138 def _score(r: Result, p: GenParams, tx_bits: np.ndarray, core: bool) -> tuple[bool, str]:
86 139     want = _EXPECT.get(p.mod, p.mod)
87 140     fsk = family(p.mod) == "fsk"
88 141     e_fc = abs(r.fc - p.fc)
89 142     e_bd = abs(r.baud - p.baud) / p.baud
90 143     checks = [r.mod == want, e_bd < 0.01, r.lock >= (50 if want.endswith("qam") else 60)]
91 144     # pi4dqpsk's 4th power rotates pi/symbol, so the carrier estimate absorbs baud/8;
92 145     # the shifted band then also biases the occupied-bandwidth rolloff. Bits stay exact.
93 146     skew = p.mod == "pi4dqpsk"
94 147     if not skew:
95 148         checks.append(e_fc < max(300, 1e-4 * p.fs))
96 149     if not fsk and not skew and p.rolloff >= 0.15 and not p.taps:
97 150         checks.append(abs(r.rolloff - p.rolloff) < 0.08)
98 151     if fsk:
99 152         checks.append(abs(r.h - (0.5 if p.mod == "msk" else p.h)) < 0.15)
100 153     b = -1.0
101 154     if core:
102 155         k = mod_order(r.mod).bit_length() - 1
103 156         if fsk:
104 157             b = ber_bits(r.bits, tx_bits, k)
105 158         elif p.mod in _EXPECT: # differential: transitions, no rotation search
106 159             b = ber_bits(demap_diff_bits(r.symbols, _DIFF[p.mod]), tx_bits[k:], k)
107 160         else:
108 161             b = ber(r.symbols, want, tx_bits)
109 162         checks.append(b <= (0.002 if (p.taps or p.mod == "fsk4") else 0.0))
110 163     extra = f"h{r.h:.2f}" if fsk else f"roll{r.rolloff:.2f}"
111 164     msg = (f"{r.mod:>6} fc{e_fc:7.1f} baud{e_bd * 100:5.2f}% {extra} lock{r.lock:5.1f}"
112 165           + (" EQ" if r.eq_applied else "") + (f" ber{b:.4f}" if b >= 0 else ""))
113 166     return all(checks), msg
114 167
115 168
116 169 def _grid(tier: str, seeds: int) -> list[tuple[str, bool, GenParams]]:
117 170     """(label, is_core, params) cases. CORE must pass 100%; STRETCH is report-only."""
118 171     fs, cases = 1e6, []
119 172     for mod in MODS:
120 173         for ratio in (10.0, 7.69):
121 174             for sd in range(seeds):
122 175                 rng = np.random.default_rng(sd)
123 176                 cases.append((f"{mod}/r{ratio:g}/s{sd}", True, GenParams(
124 177                     mod=mod, fs=fs, baud=fs / ratio, snr=_SNR[mod], fc=0.008 * fs,
125 178                     phase=rng.uniform(0, 2 * np.pi), timing=rng.uniform(), seed=sd)))
126 179     # real passband .pcm: fc > baud*(1+roll)/2 and sym*fc < fs/2 must both hold
127 180     for mod, baud, fc in (("bpsk", fs / 10, 0.1 * fs), ("qpsk", fs / 10, 0.1 * fs),
128 181                          ("8psk", fs / 20, 0.05 * fs), ("16qam", fs / 10, 0.1 * fs)):
129 182         cases.append((f"{mod}/real", True, GenParams(
130 183             mod=mod, fs=fs, baud=baud, snr=_SNR[mod], fc=fc, fmt="real", seed=0)))
131 184     cases += [("qpsk/cfo0", True, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=14, seed=1)),
132 185             ("qpsk/dc", True, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=14,
133 186                 fc=0.008 * fs, dc=0.3 + 0.3j, seed=2)),
134 187             ("qpsk/pad", True, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=14,
135 188                 fc=0.008 * fs, pad=0.5, seed=3)),
136 189             ("32qam", True, GenParams(mod="32qam", fs=fs, baud=fs / 10, snr=24,

```

```

137 190         fc=0.008 * fs, seed=0))]
138 191 for mod, h in (("fsk2", 0.7), ("fsk4", 0.7), ("msk", 0.5)): # frequency family
139 192     for sd in range(min(seeds, 2)):
140 193         cases.append((f"{mod}/s{sd}", True, GenParams(
141 194             mod=mod, fs=fs, baud=fs / 10, snr=18, h=h, seed=sd)))
142 195 for mod in ("dbpsk", "dqpsk", "pi4dqpsk"): # differential: constellation of the parent
143 196     cases.append((f"{mod}", True, GenParams(mod=mod, fs=fs, baud=fs / 10, snr=16,
144 197         fc=0.008 * fs, seed=0)))
145 198 for mod in ("qpsk", "16qam"): # symbol-spaced multipath -> equalizer rescue
146 199     cases.append((f"{mod}/multipath", True, GenParams(
147 200         mod=mod, fs=fs, baud=fs / 10, snr=18 if mod == "qpsk" else 24, fc=0.008 * fs,
148 201         taps=(1.0, 0.45 * np.exp(1j * 0.8), 0.2j), tap_sym=1.0, seed=0)))
149 202 # v3.1: 1.5-symbol echo (T/2 FSE rescue), low-rolloff baud fallback, deep alias
150 203 cases += [("qpsk/2ray-fse", True, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=18,
151 204     fc=0.008 * fs, taps=(1.0, 0.8),
152 205     tap_sym=1.5, seed=0)),
153 206     ("qpsk/roll0.05", True, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=18,
154 207     rolloff=0.05, fc=0.008 * fs, seed=0)),
155 208     ("16qam/roll0.08", True, GenParams(mod="16qam", fs=fs, baud=fs / 10, snr=24,
156 209     rolloff=0.08, fc=0.008 * fs, seed=0)),
157 210     ("qpsk/alias", True, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=14,
158 211     fc=1.3 * fs / 8, seed=0))]
159 212 if tier == "full":
160 213     for mod in MODS:
161 214         cases.append((f"{mod}/lowsnr", False, GenParams(
162 215             mod=mod, fs=fs, baud=fs / 10, snr=_SNR_LOW[mod], fc=0.008 * fs, seed=0)))
163 216     for roll in (0.1, 0.2, 0.5):
164 217         cases.append((f"qpsk/roll{roll}", False, GenParams(
165 218             mod="qpsk", fs=fs, baud=fs / 10, snr=18, rolloff=roll, fc=0.008 * fs, seed=0)))
166 219     cases += [("qpsk/bigcfo", False, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=18,
167 220         fc=0.03 * fs, seed=0)),
168 221         ("qpsk/drift", False, GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=18,
169 222         fc=0.008 * fs, drift_ppm=100, seed=0)),
170 223         ("16qam/sps4", False, GenParams(mod="16qam", fs=fs, baud=fs / 4, snr=24,
171 224         fc=0.008 * fs, seed=0))]
172 225     return cases
173 226
174 227
175 228 def _sweep(args: argparse.Namespace) -> int:
176 229     cases = _grid(args.tier, args.seeds)
177 230     fails, tally = [], {"CORE": [0, 0], "STRETCH": [0, 0]} # tier -> [pass, total]
178 231     for label, core, p in cases:
179 232         x, bits = generate(p)
180 233         try:
181 234             r = analyze(x, Meta(p.fs, p.fmt, p.dtype)) # scoring demaps separately
182 235             ok, msg = _score(r, p, bits, core)
183 236         except Exception as exc: # a crash is a failure, not an abort
184 237             ok, msg = False, f"CRASH {exc}"
185 238         tier = "CORE" if core else "STRETCH"
186 239         tally[tier][0] += ok
187 240         tally[tier][1] += 1
188 241         print(f"[{'PASS' if ok else 'FAIL'}] {tier:<7} {label:<16} {msg}")
189 242         if core and not ok:
190 243             fails.append(label)
191 244     for tier, (good, n) in tally.items():
192 245         if n:
193 246             print(f"{tier}: {good}/{n} pass")
194 247     if fails:
195 248         print(f"CORE failures: {'', '.join(fails)}")
196 249     return 1

```

```

197 250         return 0
198 251
199 252
200 253 # --- analyze / gen / serve -----
201 254
202 255 def _analyze(args: argparse.Namespace) -> int:
203 256     r = analyze_file(args.file, args.fs, args.fmt, args.dtype,
204 257                     "be" if args.be else None, args.bitrev or None, args.diff,
205 258                     None if args.burst is None else args.burst - 1, rf=args.rf)
259 +     j = r.to_json(views=False) # 한 줄 요약도 여기서 만든다 (반올림이 한 번만 걸리게)
260 +     d = j["detected"]
207 261     truth = (sidecar_read(args.file) or {}).get("truth") # display only, never detection
208 262     if len(r.bursts) > 1:
209 263         print(f"버스트 {len(r.bursts)}개 감지 - {r.burst_idx + 1}번째 분석 (--burst N 으로 선택)
264         ")
210 264         print(f"모드      {d['mod']}" + (f"      (정답 {truth['mod']})" if truth else ""))
211 265         print(f"반송파    {d['fc']:.1f} Hz" + (f"      (정답 {truth['fc']:.1f})" if truth else ""))
212 266         if d["rf_hz"] is not None:
213 267             print(f"실제 RF    {d['rf_hz'] / 1e6:.6f} MHz")
214 268         print(f"심볼레이트 {d['baud']:.1f} Hz" + (f"      (정답 {truth['baud']:.1f})" if truth else ""))
269         )
215 269         fsk = r.family == "fsk"
216 270         tail = f"변조지수 h {d['h']:.2f}" if fsk else f"롤오프    {d['rolloff']:.2f}"
217 271         mer = "" if fsk else f"MER {r.mer_db:.1f} dB"
218 272         print(f"{tail}    lock {r.lock:.1f}{mer}")
219 273         if r.eq_applied:
220 274             print("등화기 적용: 다중경로 ISI 보정됨"
221 275                 + (" (T/2 분수간격)" if r.eq_mode == "fse" else " (심볼간격)"))
222 276         if r.alias_resolved:
223 277             print("반송파 앨리어스 보정: 스펙트럼 중심으로 후보 선택")
224 278         if r.baud_fallback:
225 279             print("심볼레이트 폴백: 점유대역폭 사전정보로 재탐색")
226 280         if r.carrier_ambiguous:
227 281             print("경고: 반송파 앨리어싱 가능 (|sym*fc| > 0.4*fs)")
228 282         if args.save_iq:
229 283             r.save_iq(args.save_iq)
230 284         if args.save_symbols:
231 285             r.save_symbols(args.save_symbols)
232 286         if args.save_bits:
233 287             r.save_bits(args.save_bits, args.packed)
234 288         if args.report:
235 289             r.save_report(args.report)
290 +     if args.brief: # 사람용 출력을 대체하지 않고 맨 끝에 한 줄 더 --
291 +         print(brief(j, "an")) # 조기 return 이면 위 저장들이 조용히 무시된다
292     return 0
293
294
295 def _fhz(v: float) -> str:
296     a = abs(v)
297     if a >= 1e6:
298         return f"{v / 1e6:+.3f} MHz"
299     if a >= 1e3:
300         return f"{v / 1e3:+.2f} kHz"
301     return f"{v:+.0f} Hz"
302
303
304 def _survey(args: argparse.Namespace) -> int:
305     """Wideband survey: detect every emitter, demodulate the digital ones."""
306     s = survey_file(args.file, args.fs, args.fmt, args.dtype,
307                     "be" if args.be else None, args.bitrev or None, args.diff, rf=args.rf)

```

```

252 308 rf0 = s.meta.rf_center
253 309 note = f" · RF 중심 {rf0 / 1e6:.3f} MHz" if rf0 is not None else ""
254 310 print(f"{len(s.emitters)}개 신호 감지 (샘플레이트 {s.meta.fs:.0f} Hz{note})")
255 311 print(f"{'#':>2} {'실제 RF' if rf0 is not None else '중심주파수'}:>12} {'대역폭':>10}"
256 312 f" {'분류':>7} {'변조':>7} {'심볼레이트':>11} {'lock':>5}")
257 313 for i, e in enumerate(s.emitters):
258 314     r = e.result
259 315     mod = r.mod if r else "-"
260 316     baud = f"{r.baud:.0f}" if r else "-"
261 317     lock = f"{r.lock:.0f}" if r else "-"
262 318     kind = {"linear": "디지털", "fsk": "FSK", "analog": "아날로그",
263 319           "tone": "순수톤", "tooshort": "너무짧음", "error": "오류"}.get(e.kind, e.kind)
264 320     fc = e.abs_fc if rf0 is None else rf0 + e.abs_fc
265 321     print(f"{'i':>2} {_fhz(fc):>12} {_fhz(e.detection.bw):>10} {kind:>7}"
266 322           f" {mod:>7} {baud:>11} {lock:>5}")
267 323 if args.report:
268 324     import json
269 325     with open(args.report, "w") as fh:
270 326         json.dump(s.to_json(), fh, indent=1)
327 + if args.brief:
328 +     print(brief(s.to_json(), "sv"))
271 329 return 0
272 330
273 331
274 332 def _gen(args: argparse.Namespace) -> int:
275 333     if args.fmt == "real" and args.cfo <= args.baud * (1 + args.rolloff) / 2:
276 334         print("real 포맷은 통과대역 반송파가 필요합니다: --cfo > baud*(1+rolloff)/2")
277 335         return 1
278 336     taps = tuple(complex(t) for t in args.taps.split(",")) if args.taps else ()
279 337     p = GenParams(mod=args.mod, n_symbols=args.n, fs=args.fs, baud=args.baud,
280 338                 rolloff=args.rolloff, fc=args.cfo, phase=args.phase, timing=args.timing,
281 339                 snr=args.snr, fmt=args.fmt, dtype=args.dtype,
282 340                 endian="be" if args.be else "le", bitrev=args.bitrev, seed=args.seed,
283 341                 pad=args.pad, drift_ppm=args.drift_ppm, dc=complex(args.dc),
284 342                 h=args.h, taps=taps)
285 343     print(save(p, args.out, args.label))
286 344     return 0
287 345
288 346
289 347 def _dataset(args: argparse.Namespace) -> int:
290 348     """Curated variation matrix: every modulation + sample-type/endian/bitrev
291 349     variants + impairment cases, all with ground-truth sidecars."""
292 350     fs, out, made = 1e6, args.out, []
293 351     for mod in GEN_MODS: # one canonical file per modulation
294 352         fc = 0.0 if family(mod) == "fsk" else 0.008 * fs
295 353         h = 0.7 if mod == "fsk2" else 0.5 # fsk2 at h=0.5 IS msk; pick a distinct index
296 354         made.append(save(GenParams(mod=mod, fs=fs, baud=fs / 10, snr=22, fc=fc, h=h,
297 355                                 phase=0.6, timing=0.3, seed=1), out, mod))
298 356     # sample-type variants: label must stay unique (endian/bitrev alone would collide)
299 357     for i, (dt, be, br) in enumerate(((("i8", 0, 0), ("u8", 0, 0), ("u16", 0, 0),
300 358                                     ("f32", 0, 0), ("f64", 0, 0), ("i16", 1, 0),
301 359                                     ("i16", 0, 1), ("u8", 1, 1))):
302 360         made.append(save(GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=18,
303 361                                 fc=0.008 * fs, dtype=dt, endian="be" if be else "le",
304 362                                 bitrev=bool(br), seed=2), out, f"formatvar{i}"))
305 363     for mod in ("bpsk", "qpsk", "16qam"): # real passband .pcm
306 364         made.append(save(GenParams(mod=mod, fs=fs, baud=fs / 10, snr=18, fc=0.1 * fs,
307 365                                 fmt="real", seed=3), out, f"pcm-{mod}"))
308 366     impair = [{"dc", {"dc": 0.3 + 0.3j}}, {"pad", {"pad": 0.5}},
309 367              {"drift", {"drift_ppm": 100}}, {"lowsnr", {"snr": 8}},

```

```

310 368         ("taps", {"taps": (1.0, 0.45 * np.exp(1j * 0.8), 0.2j)}]) # 1-symbol echoes
311 369     for label, kw in impair:
312 370         made.append(save(GenParams(mod="qpsk", fs=fs, baud=fs / 10, snr=kw.pop("snr", 18),
313 371                               fc=0.008 * fs, seed=4, **kw), out, label))
314 372     made.append(save(GenParams(mod="fsk4", fs=fs, baud=fs / 10, snr=20, h=1.0, seed=5),
315 373                     out, "fsk4h1"))
316 374     if len(set(made)) != len(made):
317 375         print("경고: 파일명 충돌 - 라벨이 겹칩니다")
318 376     return 1
319 377     print(f"{len(made)} files -> {out}/ (각 파일에 <이름>.json 정답 사이드카)")
320 378     return 0
321 379
322 380
323 381 def _read_args(p: argparse.ArgumentParser) -> None:
324 382     """Read-side sample-format overrides, shared by analyze/survey (sidecar/filename win)."""
325 383     p.add_argument("--fs", type=float)
326 384     p.add_argument("--fmt", choices=("iq", "real"))
327 385     p.add_argument("--dtype", choices=_DTYPE_CHOICES)
328 386     p.add_argument("--be", action="store_true", help="big-endian 샘플")
329 387     p.add_argument("--bitrev", action="store_true", help="바이트 내 비트 역순")
330 388     p.add_argument("--rf", type=float, help="RF 중심주파수 (Hz) - 실제 주파수로 보고")
331 389 +     p.add_argument("--brief", action="store_true",
332 390 +                   help="손으로 옮길 한 줄 + 오타 검출 코드를 끝에 덧붙임")
333 391
334 392
335 393 def main(argv: list[str] | None = None) -> int:
336 394     ap = argparse.ArgumentParser(prog="signus")
337 395     sub = ap.add_subparsers(dest="cmd", required=True)
338 396
339 397     a = sub.add_parser("analyze", help="블라인드 복조 + 제원 탐지")
340 398     a.add_argument("file")
341 399     _read_args(a)
342 400     a.add_argument("--save-iq")
343 401     a.add_argument("--save-symbols")
344 402     a.add_argument("--save-bits")
345 403     a.add_argument("--packed", action="store_true")
346 404     a.add_argument("--report")
347 405     a.add_argument("--diff", action="store_true",
348 406                   help="차동 디맵(D-BPSK/D-QPSK): 회전 모호성 없이 비트 복원")
349 407     a.add_argument("--burst", type=int, help="분석할 버스트 번호 (1부터; 기본 최강 버스트)")
350 408
351 409     sv = sub.add_parser("survey", help="광대역 캡처의 모든 신호 탐지 + 복조")
352 410     sv.add_argument("file")
353 411     _read_args(sv)
354 412     sv.add_argument("--diff", action="store_true", help="차동 디맵")
355 413     sv.add_argument("--report", help="JSON 리포트 저장 경로")
356 414
357 415     g = sub.add_parser("gen", help="합성 신호 + 정답 사이드카 생성")
358 416     g.add_argument("--mod", default="qpsk", choices=GEN_MODS)
359 417     g.add_argument("--fs", type=float, default=1e6)
360 418     g.add_argument("--baud", type=float, default=1e5)
361 419     g.add_argument("--snr", type=float, default=20)
362 420     g.add_argument("--rolloff", type=float, default=0.35)
363 421     g.add_argument("--cfo", type=float, default=0.0)
364 422     g.add_argument("--phase", type=float, default=0.0)
365 423     g.add_argument("--timing", type=float, default=0.0)
366 424     g.add_argument("--pad", type=float, default=0.0)
367 425     g.add_argument("--drift-ppm", type=float, default=0.0)
368 426     g.add_argument("--dc", default="0")
369 427     g.add_argument("--h", type=float, default=0.5, help="FSK 변조지수")

```

```

368 428 g.add_argument("--taps", default="", help="멀티패스 탭, 예: 1,0,0.35j")
369 429 g.add_argument("--fmt", default="iq", choices=("iq", "real"))
370 430 g.add_argument("--dtype", default="i16", choices=_DTYPE_CHOICES)
371 431 g.add_argument("--be", action="store_true")
372 432 g.add_argument("--bitrev", action="store_true")
373 433 g.add_argument("--n", type=int, default=6000)
374 434 g.add_argument("--seed", type=int, default=0)
375 435 g.add_argument("--out", default="samples")
376 436 g.add_argument("--label", default="sig")
377 437
378 438 d = sub.add_parser("dataset", help="변조x샘플타입x장래 변형 매트릭스 일괄 생성")
379 439 d.add_argument("--out", default="samples")
380 440
381 441 w = sub.add_parser("sweep", help="전수조사: 그리드 생성->복조->채점")
382 442 w.add_argument("--tier", default="core", choices=("core", "full"))
383 443 w.add_argument("--seeds", type=int, default=3)
384 444
385 445 s = sub.add_parser("serve", help="웹 UI 서버")
386 446 s.add_argument("--host", default="127.0.0.1")
387 447 s.add_argument("--port", type=int, default=8000)
388 448
389 449 args = ap.parse_args(argv)
390 450 if args.cmd == "analyze":
391 451     return _analyze(args)
392 452 if args.cmd == "survey":
393 453     return _survey(args)
394 454 if args.cmd == "gen":
395 455     return _gen(args)
396 456 if args.cmd == "dataset":
397 457     return _dataset(args)
398 458 if args.cmd == "sweep":
399 459     return _sweep(args)
400 460 from .server import run
401 461 run(args.host, args.port)
402 462 return 0
403 463
404 464
405 465 if __name__ == "__main__":
406 466     sys.exit(main())

```